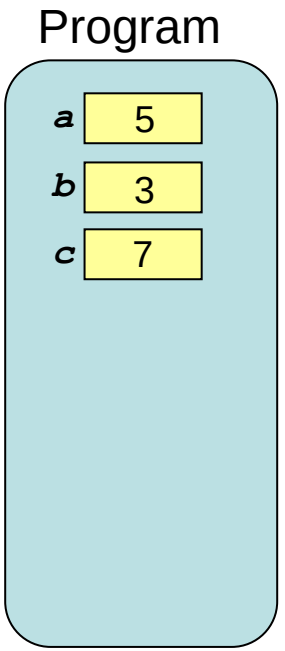
Variables

```
#include <stdio.h>

int main
{
    int a, b, c;

    a=10;
    b=3;
    c=a-b;
    a=b+2;
}
```



Rules for identifier names

- While defining names for variables (and also functions, user-defined types, and constants in the future) you should obey the following rules:
  - The first character of a name must be a letter or underscore ('_').
  - The remaining characters must be letters, digits, or underscore.
  - Only the first 31 characters are significant.
  - Avoid reserved words such as int, float, char, etc. as identifier names.
- However, it is better to avoid starting identifier names with underscore.
- Also remember that C language is case-sensitive.
- It is a very good practice to use meaningful names.

Rules for identifier names

- Valid:

```
a, a1, count, no_of_students, B56, b_56
```

- Invalid:

```
1a, say1, int, $100
```

- Valid but not recommended:

```
_b56, Salih, FB, GS, BJK,
I_dont_remember_what_this_variable_means,
a_very_very_long_identifier_name_1,
a_very_very_long_identifier_name_2
```

Standard data types

- You have to specify the type of a variable when you define it.
- There are three standard data types:
  - Integer (i.e., whole numbers)
  - Float (i.e., real or floating-point numbers)
  - Characters
- We will discuss user-defined types later in the course.

Integers

- Syntax:

```
int variable_list;
```

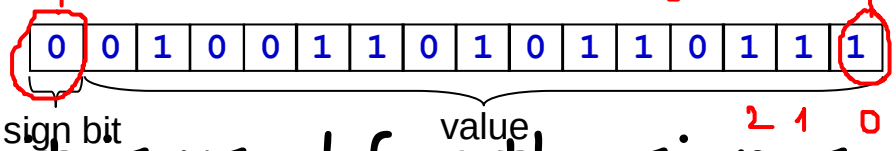
where `variable_list` is a comma-separated list of variable names. Each variable name may be followed by an optional assignment operator and a value for initialization.

-Eg: `int a, b=10, c;`

- Integer is a class of variable types. The most basic one is `int`.
- The size may change, but the leftmost bit is used for the sign. The remaining bits represent the value in binary.
- Though the size of an int variable may vary, it is always limited, i.e., it contains a limited number of bits. Therefore, the maximum and minimum values that can be represented by an int variable is limited.

Integers

- For example, assume in your system an integer has 16 bits.



- Leftmost bit is used for the sign, so 15 bits are left for the value. So, you have 2^{15}=32,768 positive values, ranging from 0 to 32,767. Similarly, you have 32,768 negative values, this time ranging from -1 to -32,768.
- If you have 32 bits (4 bytes) for an integer, than the maximum value is 2^{31}=2,147,483,647.

Integers

sizeof

- There are variations of **`int`** such as **`long int`**, **`short int`**, **`unsigned int`**.
  - For each one of these types, you may ignore the word "int" and use **`long`**, **`short`**, and **`unsigned`**, respectively.
- The sizes of these types are ordered as follows:

  short int ≤ int ≤ long int

Floating-point numbers

- Syntax:

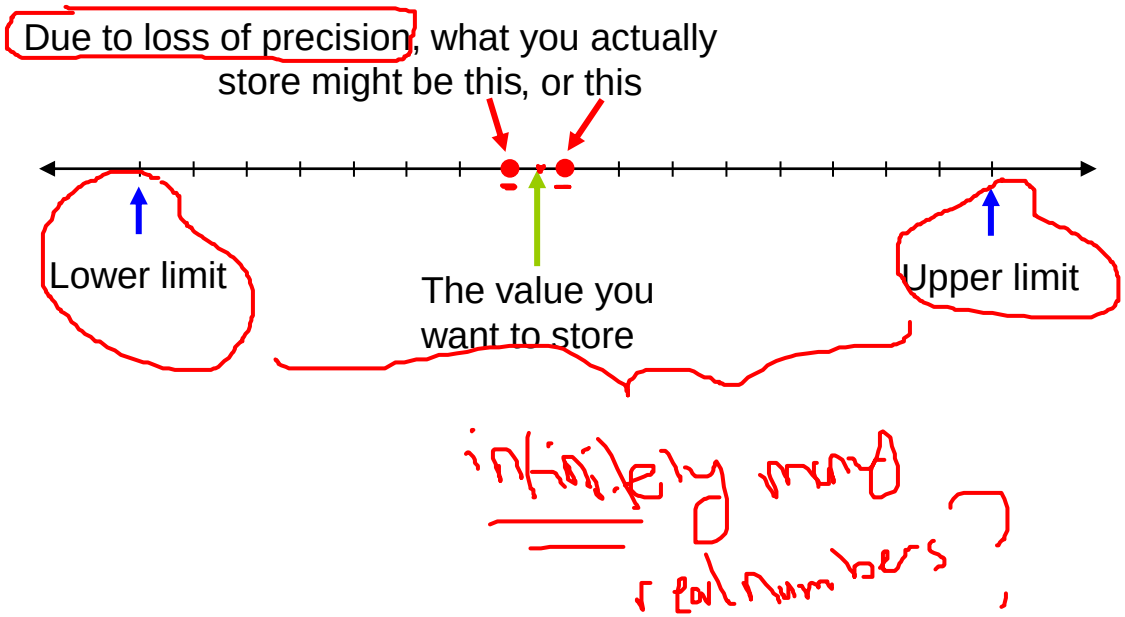
```
float variable_list;
```

- Float type is used for real numbers.
- Note that all integers may be represented as floating-point numbers, but not vice versa.



Floating-point numbers

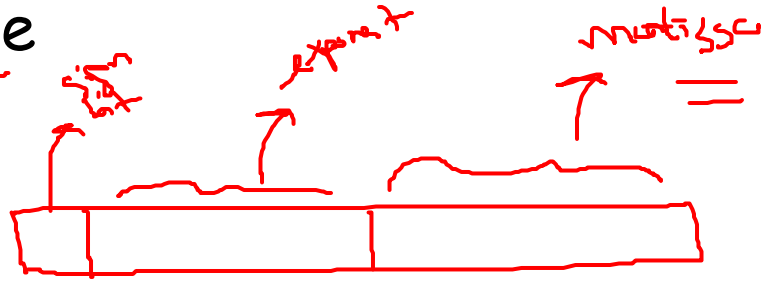
- Similar to integers, floats also have their limits: maximum and minimum values are limited as well as the precision.



Floating-point numbers

- There are two variations of **`float`**: **`double`** and **`long double`**.
  - They have wider range and higher precision.
- The sizes of these types are ordered as follows:

  float ≤ double ≤ long double



Characters

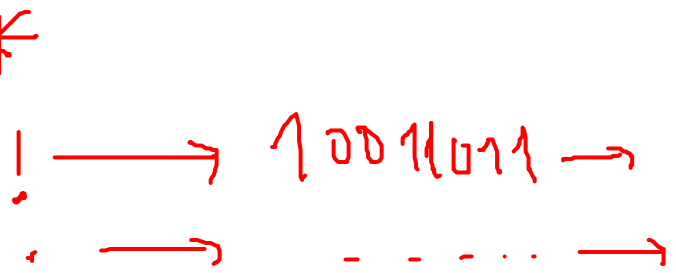
- Syntax:

```
char variable_list;
```

- Character is the only type that has a fixed size in all implementations: 1 byte.
- All letters (uppercase and lowercase, separately), digits, and signs (such as +,-,!,?,$,£,^,#, comma itself, and many others) are of type character.

Characters

- Since every value is represented with bits (0s and 1s), we need a mapping for all these letters, digits, and signs.
- This mapping is provided by a table of characters and their corresponding integer values.
  - The most widely used table for this purpose is the ASCII table.



Characters

- The ASCII table contains the values for 256 values (of which only the first 128 are relevant for you). Each row of the table contains one character. The row number is called the ASCII code of the corresponding character.

  (The topic of character encoding is beyond the scope of this course. So, we will work with the simplified definition here.)

Characters

- Never memorize the ASCII codes. They are available in all programming books and the Internet. (Eg: http://www.ascii-code.com)
- What is important for us is the following three rules:
  - All lowercase letters (a,b,c,...) are consecutive.
  - All uppercase letters (A,B,C,...) are consecutive.
  - All digits are consecutive.

ASCII table (partial)

ASCII code	Symbol	ASCII code	Symbol	ASCII code	Symbol	ASCII code	Symbol
...	...	66	B	84	T	107	k
32	blank	67	C	85	U	108	l
37	%	68	D	86	V	109	m
42	*	69	E	87	W	110	n
43	+	70	F	88	X	111	o
...	...	71	G	89	Y	112	p
48	0	72	H	90	Z	113	q
49	1	73	I	...	...	114	r
50	2	74	J	97	a	115	s
51	3	75	K	98	b	116	t
52	4	76	L	99	c	117	u
53	5	77	M	100	d	118	v
54	6	78	N	101	e	119	w
55	7	79	O	102	f	120	x
56	8	80	P	103	g	121	y
57	9	81	Q	104	h	122	z
...	...	82	R	105	i	...	...
65	A	83	S	106	j		

Characters

- A character variable actually stores the ASCII value of the corresponding letter, digit, or sign.
- I/O functions (printf(), scanf(), etc.) do the translation between the image of a character displayed on the screen and the ASCII code that is actually stored in the memory of the computer.

Characters

- Note that `a` and `A` have different ASCII codes (`97` and `65`).
- You could also have a variable with name `a`. To differentiate between the variable and the character, we specify all characters in single quotes, such as `'a'`. Variable names are never given in quotes.
  - Example:
```
char ch;
ch='a';
```
- Note that using double quotes makes it a string (to be discussed later in the course) rather than a character. Thus, `'a'` and `"a"` are different.
- Similarly, `1` and `'1'` are different. Former has the value `1` whereas the latter has the ASCII value of `49`.

Characters

- Example: Consider the code segment below.

```
char ch;
ch='A';
printf("Output is %c", ch);
```

- The string in printf() is stored as

```
79,117,116,112,117,116,32,105,115,32,37,99
```

  which are the ASCII codes of the characters in the string.

- When printf() is executed, it first replaces **37,99** (**%c**) with **65** (**A**), and then displays the corresponding characters on the screen.